DEEP-DIVE ARCHITECTURAL & LOGIC AUDIT #05E: NON-WEAPON EQUIPMENT & PROCEDURAL SYSTEM (PART 5 — HOLISTIC OVERVIEW OF ALL NON-WEAPON EQUIPMENT: 333 ARMORS, ACCESSORIES, TOOLS, TALISMANS & CORES)

- **Target Codebase**: `JianghuSect 2` (`JS2-v2.6.0` / `SIM.PATCH_VERSION = '2.6.0'` / `HC.FINGERPRINT = '155:2f9c9dd7'`)
- **Target Audience**: Autonomous AI Software Engineers (**GPT** & **Gemini** Models)
- **Master Roadmap (`05A–05G`)**:
  - `05A–05D`: Granular Group Audits (`Body Armor`, `Jewelry/Shoes/Pouches`, `Talismans/Cores/Tools`, `Resonance Sets/Pairs/Catalysts`)
  - **`05E` (This Document)**: **Holistic Overview of All Non-Weapon Equipment (`117` Armors + `216` Accessories = `333` Non-Weapon Items)** — Cross-cutting structural bottlenecks that emerge only when viewing all non-weapon categories together!
  - `05F`: Part 6 — Penultimate Deep Audit: Procedural Equipment Engine, `ITEM_PROC_*`/`REC_PROC_*` Templates & Related Systems
  - `05G`: Part 7 — Grand Finale: Master End-to-End Audit of the Entire Equipment Ecosystem (`898` Items Total)

---

0. Hard Architectural Invariants for AI Implementers

1. **Immutable `human_core.js`**: Never modify `GAME_SOURCE/js/human_core.js` (`HC.FINGERPRINT === '155:2f9c9dd7'`).
2. **Preserve 3-Slot Set Contracts**: Keep `WEAPON`, `ARMOR`, and `ACCESSORY` as the canonical 3 combat/resonance slots so all `62` Resonance Sets and `3/3` Primordial Trinity contracts remain unbroken.
3. **100% Emergent Sect Progression**: As disciples advance from Stage 1 to Stage 7, the sect's Forge, Array, and Talisman halls must naturally upgrade from Tier-1 starter tunics to Tier 4–7 vestments, tools, and charms—without getting stuck at `"noArmor === 0"` forever!

---

1. [NWEQ-SLOT-01] The Single-ACCESSORY-Slot Congestion Crisis: 216 Items Across 9 Incompatible Functional Classes Crammed Into 1 Slot!

Out of 898 equipment items (565 Weapons, 117 Armors, 216 Accessories), **one single `npc.equip.ACCESSORY` slot** is forced to hold **nine physically and functionally unrelated item categories**:

1. Finger Rings (`RING`)
2. Necklaces & Pendants (`NECKLACE` / `PENDANT`)
3. Wrist Bracelets (`BRACELET`)
4. Waist Jades & Sachets (`CHARM` / `ORNAMENT`)
5. Cloud-Step Footwear (`SHOES` — *wearing a Ring forces the cultivator to take off their boots!*)
6. Spatial Logistics Pouches (ถุงเฉียนคุนจัดสรรเสบียง)
7. Paper Combat Talismans (`ITEM190_TAL_001..024` — 24 expendable talismans)
8. Portable Array Pillars, Siege Spikes & Banners (`ITEM190_CORE_001..012`)
9. Heavy Workshop Cauldrons, Smithing Hammers, Herb Brushes & Beast Flutes (`ITEM190_TOOL_001..006`, `REC140_FRG_MITHRIL_CAULDRON_4`)

Chain-Reaction Simulation Failure:

- `M.gearSuitability` (`js/martial.js:539–608`) scores **every** `ACCESSORY` item on a single combat-dominated scale (`atk + matk + def + mdef + hp*0.5 + crit*8 + pen*6`).
- Consequently:
  - Any Tier-4+ combat Ring/Charm (`score 320–850`) **permanently crowds out** all Workshop Tools (`Mithril Cauldron`, `Jade Fire-Tongs`), Spatial Pouches, and Tribulation Cores (`score 90–190`).
  - Conversely, NPCs have **zero contextual gear-switching AI** (an Alchemist never equips a Pill Cauldron at the furnace and swaps back to a Protective Jade Charm before departing on a raid!).

Clean Zero-Save-Breakage Architectural Fix:

Keep `n.equip = { WEAPON, ARMOR, ACCESSORY }` untouched for combat stats & 3-piece sets, and introduce **Contextual Bag/Bench Utility Resolution**:

1. **Workshop / Farm / Taming Tools (`cat === 'TOOL'` or `craftBonusPath`)**: `ECO.equippedToolBonus(npc, domain)` checks `npc.equip.ACCESSORY` **first**, and if occupied by combat jewelry, automatically uses **the highest-tier matching usable tool in `npc.inv.gear` (personal toolbag!)** during `brew`, `forge`, `farm`, or

`tame`—and teach crafter NPCs (`COG` / `CPE`) to keep **1 primary work tool** in `npc.inv.gear` (`category: 'valuable-to-own'`)!
2. **Logistics Pouches & Tribulation/Siege Cores**: During `EXPED.depart`, `CULT.attemptBreakthrough`, or `EXPED.resolveSectSiege`, check `npc.equip.ACCESSORY` **or 1 carried utility core/pouch in `npc.inv.gear`**.
3. **Paper Talismans (`cat === 'TALISMAN'`)**: Route into `npc.armedConsumables` (`max 2 belt talismans`) rather than displacing permanent `ACCESSORY` jewelry!

---

2. `[NWEQ-MULT-02]` Major Engine Discovery: `it.combatMult` & `it.defMult` Are Computed Everywhere in `economy.js` & Scored by AI, Yet `100%` Ignored by `stats.js` & `combat.js`!

- **Locations**: `js/economy.js:221, 239, 244, 531–532, 709–710`, `js/martial.js:541–544` vs `js/stats.js:365–377, 595–605`
- **Verified Proof**:
    1. Every minted and procedural armor/accessory/weapon calculates `combatMult` and `defMult` (`e.g. REC140_FRG_IMMORTAL_ARMOR_6` sets `defMult: 0.24` (`+24%` defense!); `ECO.applyArmorArtisan` boosts `gear.defMult *= (1 + bonus)`; `ECO.refineGearWithCatalyst` scales `combatMult` and `defMult` by `ratio`).
    2. In `M.gearSuitability` (`js/martial.js:541–544`), **NPCs explicitly add large suitability points for `it.combatMult` and `it.defMult`**:

```JavaScript
if (it.combatMult || it.defMult) {
  const stageScale = 22 + c.stage * 16;
  power += (it.combatMult || 0) * stageScale * (role.atk + role.matk) * .55
         + (it.defMult || 0) * stageScale * (role.def + role.mdef) * .55;
}
```

    3. **Yet in `js/stats.js` (`compute` lines 365–377 AND `getBaseFast` lines 595–605), ONLY `ST.gearStats(it)` (`gs[key] * df`) is added—`it.combatMult` and `it.defMult` are NEVER read anywhere in `stats.js` or `combat.js` (`0.00%` actual combat effect)**!
- **Impact**: Recipe-authored armors and cores whose power budget was authored primarily in `effect.defenseMult` (`REC140_FRG_IMMORTAL_ARMOR_6` `0.24`, `REC140_ARR_ASTRAL_CORE_5` `0.18`) trick NPCs into paying high CP/stone prices while delivering **zero** percentage defense in battle!

- **Fix**: Either apply a bounded fraction of `it.combatMult` / `it.defMult` (`e.g. * 0.35 * df` as `addP('atk'/'matk')` and `addP('def'/'mdef')` in both `compute` and `getBaseFast`, calibrated with `combat_balance.js`), **or** fold `effect.defenseMult` / `effect.combatMult` on `REC140_*` recipe-only items directly into `ST.gearStats` so `M.gearSuitability` never scores phantom stats!

---

3. `[NWEQ-STAT-03]` Three Stat-Formula Anomalies in `ST.gearStats` (`js/stats.js:44–92`) for Authored Armors & Accessories

1. **40% of Authored Accessories Roll Zero Primary Stats (`stats.js:56`)**:
    - Look at `stats.js:56` for non-procedural `ACCESSORY` items without `fixedStats`:

```
JavaScript
else { pickAff(2); const r = rn(h, 4); if (r < 0.3) s.atk = (s.atk || 0) + base * 0.2; else if (r < 0.6) s.matk = (s.matk || 0) + base * 0.2; }
```

    - Whereas procedural accessories (`economy.js:677–679`) **always** receive `{ hp: 0.28*base, atk: 0.14*base, matk: 0.14*base }` **plus** typed base **plus** 1–3 affixes, **`40%` (`r >= 0.6`) of authored accessories receive `0` `hp`, `0` `atk`, and `0` `matk`**—making many Tier-4 authored accessories weaker than Tier-2 procedural accessories! Give authored accessories the same baseline `{ hp: 0.24*base }` + role-appropriate `atk`/`matk`/`def`/`mdef` floor.
2. **Authored `ROBE` Gets Heavy Physical `def = 0.55*base` (`stats.js:55`)**:
    - Line 55 gives all authored `ARMOR`s `s.def = base * 0.55, s.mdef = base * 0.28` regardless of whether `type` is `HEAVY`, `LIGHT`, or `ROBE` (unlike `economy.js:673–674` where `HEAVY` = `0.62 def / 0.28 mdef`, `LIGHT` = `0.46 def / 0.28 mdef`, `ROBE` = `0.38 def / 0.44 mdef`). Align `stats.js:55` with `economy.js:673–674`!
3. **Map Movement Speed (`AI.stepMove`) Ignores Equipped Armor Weight & Cloud-Shoes**:
    - Verify that out-of-combat sect-map walking (`M.derived(n).walkSpeed` in `martial.js:183`) reflects `SHOES` (`+walkSpeed`) and ultra-heavy armor encumbrance slightly, rather than only affecting in-combat `ST.get(n).move`!

---

4. [NWEQ-FORGE-04] Why Sects Stop Crafting Higher-Tier Armors, Accessories, Cores & Talismans (ECO.needSummary & ECO.pickOrder, js/economy.js:411–463)!

Inspect js/economy.js:411–455 (ECO.needSummary & ECO.pickOrder):

```JavaScript
if (!n.equip || !n.equip.ARMOR) noArmor++;
if (!n.equip || !n.equip.ACCESSORY) noAccessory++;
...
const crafts = hall === 'alchemy' ? ['ALCHEMY'] : ['FORGING', 'ARRAY',
'TALISMAN'];
s += (slot === 'WEAPON' ? gapW * 0.25 : slot === 'ARMOR' ? gapA * 0.25 :
Math.min(1.3, 0.2 + gapAcc * 0.16)) - gcount * 0.15;
s -= r.tier > 3 ? (r.tier - 3) * 0.12 : 0;
```

- **Three Cascading Macro-Simulation Bottlenecks**:
  1. **"Tier-1 Rag Satisfaction" Blindness (`needSummary`, `line 416`)**:
     - `noArmor` and `noAccessory` count **only completely empty slots (`!n.equip.ARMOR`)**!
     - Once every disciple puts on a **Tier-1 Common Cloth Shirt (`tier = 1`)**, `need.noArmor === 0` and `need.noAccessory === 0` **for the rest of eternity**—even when 25 Elders have broken through to **Stage 5–7** and are still fighting Tier-6 Demon Lords in Tier-1 starter rags!
  2. **Explicit High-Tier Recipe Penalty (`s -= (r.tier - 3) * 0.12`, `line 454`)**:
     - Instead of preferring higher-tier recipes as the sect's Elders advance in stage, `ECO.pickOrder` **subtracts `-0.12` to `-0.48` from Tier 4–7 equipment recipes**, biasing the Forge to spam cheap Tier 1–3 junk forever!
  3. **`FORGING` + `ARRAY` + `TALISMAN` Jammed Into One Queue With `gapAcc` Capped at `1.3`**:
     - `ARRAY` cores and `TALISMAN` charms share the single `'forge'` hall order with `FORGING` weapons/armors and have their demand score capped at `1.3` (`Math.min(1.3, ...)`).
- **Exact Fix**:
  - In `ECO.needSummary(W)`, also compute **`undergearedArmor` and `undergearedAccessory`** (counting cultivators whose equipped item has `(it.tier || 1) + 1 < c.stage` or `it.broken || it.dur < 35`), and match recipe tier `r.tier` against the stage distribution of undergeared sect

members (`+0.35` bonus when `r.tier` upgrades an undergeared Elder/Core disciple, replacing the blind `-0.12 * (r.tier - 3)` penalty)!
- Give `ARRAY` (`arraymaster`) and `TALISMAN` scribes their own secondary order slot (`W.sect.arrayOrder`) so array cores and talismans are produced alongside weapons and armors!

---

5. `[NWEQ-CYCLE-05]` Hand-Me-Down Lifecycle & Broken-Gear Equip Guard (`EQ-12`)

2. **Block Broken/Retired Non-Weapon Equipping (`sect.js` & `ui.js`)**: Ensure `SECT.EDITOR.equipPersonalGear` and all autonomous equip paths reject `it.broken || it.retired || (it.procedural && it.dur <= 0)` so a broken armor/accessory never unseats a working piece!
3. **Active Hand-Me-Down Circulation**: When a senior disciple upgrades their `ARMOR` or `ACCESSORY`, ensure their displaced usable Tier 2–5 piece in `n.inv.gear` is promptly gifted (`COG.giveGearGift` to an undergeared `apprentice`/`spouse`) or donated to `W.sect.gear` for CP (`CPE`), keeping sect armories circulating naturally across generations!

---

6. Implementation & Verification Checklist for GPT / Gemini (`05E`)

- [ ] **Resolve `ACCESSORY` Slot Congestion**: Allow `TOOL`s, Spatial Pouches, and Tribulation/Siege Cores to function from `npc.inv.gear` (1 active utility per domain) during work/expeditions/tribulations, and route `TALISMAN`s into `armedConsumables` belt slots.
- [ ] **Reconcile `combatMult` / `defMult` (`stats.js` vs `martial.js:541`)**: Wire bounded `combatMult`/`defMult` into `ST.get` & `ST.explain` (or fold into `ST.gearStats`) so `M.gearSuitability` never evaluates phantom stats.
- [ ] **Fix `ST.gearStats` Non-Weapon Baselines (`stats.js:55–56`)**: Differentiate authored `HEAVY`/`LIGHT`/`ROBE` bases and guarantee primary stat floors on authored `ACCESSORY` items.
- [ ] **Fix `ECO.needSummary` & `ECO.pickOrder` (`economy.js:411–463`)**: Count under-tiered (`it.tier + 1 < c.stage`) and worn/broken armors/accessories as real demand, reward stage-matched `r.tier` upgrades instead of penalizing `r.tier > 3`, and separate `ARRAY`/`TALISMAN` orders from `FORGING` queue starvation.